

Internship Task (Week 4-6)

Developers Hub Corporation

CyberGuard SecOps Security Monitoring Report

Advanced Threat Detection, Web/API Security, Ethical Testing, and Final Security Audit

Project	CyberGuard SecOps Security Monitoring
Submission type	Implementation report with embedded test evidence
Target environment	Localhost API demonstration on Windows
Evidence basis	Screenshots captured during local execution and report generation
Repository	cyberguard-secops-security-monitoring

1. Executive Summary

This report presents the completed CyberGuard SecOps Security Monitoring project for the Cybersecurity Interns Task covering Weeks 4 to 6. The report follows the task structure directly: Week 4 covers API hardening, failed-login monitoring and security headers; Week 5 covers ethical security testing, SQL injection mitigation approach and CSRF protection; Week 6 covers security audit reporting, deployment-readiness artefacts and final evidence. The evidence included in this report is taken from the local execution screenshots supplied during testing. The report avoids claiming results that are not shown in the evidence.

2. Project Overview

CyberGuard is a lightweight local security demonstration consisting of two main components: **a secured Node.js/Express API** and **a Python SecOps CLI scanner**. The API implements security controls such as API key authentication, rate limiting, CORS configuration, security headers, failed-login monitoring and CSRF protection. The Python CLI scans a selected target and generates JSON and Markdown audit reports.

- The secured API provides a local test web application for demonstrating hardening and monitoring controls.
- The CLI scanner demonstrates target-based audit report generation against the local API.
- The screenshots in this report show the tested behaviours and generated report outputs.

Name	Date modified	Type	Size
.github	06-Jun-26 12:08 AM	File folder	
api	07-Jun-26 11:34 PM	File folder	
configs	05-Jun-26 9:40 PM	File folder	
docs	08-Jun-26 4:20 PM	File folder	
New folder	09-Jun-26 4:05 AM	File folder	
reports	07-Jun-26 10:57 PM	File folder	
secops_cli	07-Jun-26 11:34 PM	File folder	
.gitignore	07-Jun-26 10:48 PM	GITIGNORE File	1 KB
docker-compose.yml	06-Jun-26 12:19 AM	YML File	1 KB
Dockerfile	06-Jun-26 12:17 AM	File	1 KB
package.json	07-Jun-26 10:48 PM	JSON File	1 KB
README.md	08-Jun-26 4:20 PM	MD File	12 KB

Figure 1. Project folder structure showing the key folders and files used for submission.

Observed result: The local project contains folders such as .github, api, configs, docs, reports and secops_cli, plus README and deployment files.

3. Scope of This Report

The report is limited to the work demonstrated by the submitted project files and the screenshots provided as evidence. It is not written as a separate general cybersecurity methodology report; it is written specifically for the internship task requirements.

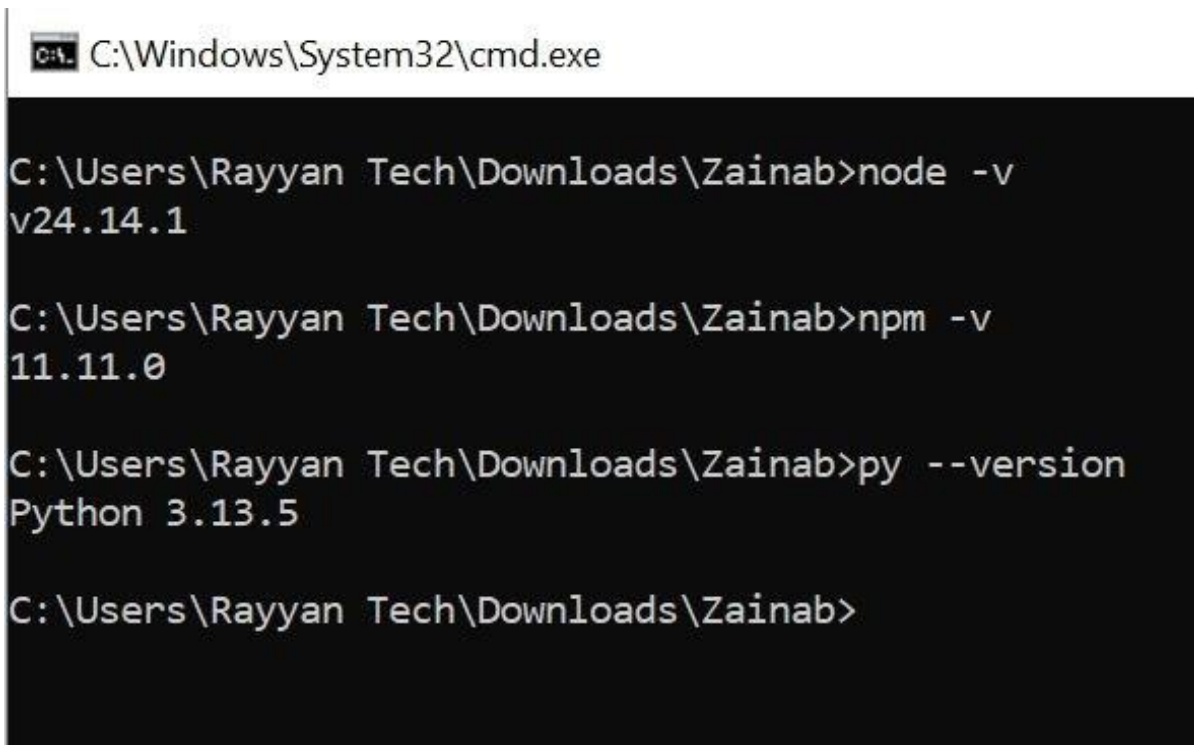
Area	Included in this report	Evidence type
Week 4	API hardening, rate limiting, failed-login monitoring, API key protection, security headers	Browser and curl screenshots
Week 5	Local security testing, SQL injection mitigation approach, CSRF implementation and testing	HTTP request/response screenshots
Week 6	CLI audit report generation, generated Markdown/JSON reports, repository/deployment-readiness artefacts	Terminal, folder and report screenshots

4. Week 4 - Advanced Threat Detection and Web Security Enhancements

Week 4 required implementation of monitoring, API hardening and security headers. The evidence below is placed directly next to the feature it supports.

4.1 Environment Validation and API Start-up

The local environment was checked first to confirm that Node.js, npm and Python were available. The project dependencies were then installed and the smoke test confirmed that the API could start and respond successfully.



```

C:\Windows\System32\cmd.exe

C:\Users\Rayyan Tech\Downloads\Zainab>node -v
v24.14.1

C:\Users\Rayyan Tech\Downloads\Zainab>npm -v
11.11.0

C:\Users\Rayyan Tech\Downloads\Zainab>py --version
Python 3.13.5

C:\Users\Rayyan Tech\Downloads\Zainab>

```

Figure 2. Environment versions for Node.js, npm and Python.

Observed result: The environment check confirms that Node.js, npm and Python were available before executing the project.

```

C:\Users\Rayyan Tech\Downloads\Zainab>npm install
> cyberguard-secops-project@1.0.0 postinstall
> npm install --prefix api

npm warn deprecated csurf@1.11.0: This package is archived and no longer maintained. For support, visit https://github.com/expressjs/express/discussions
added 155 packages, and audited 156 packages in 11s

25 packages are looking for funding
  run `npm fund` for details

2 low severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.

up to date, audited 1 package in 16s

found 0 vulnerabilities
npm notice
npm notice New minor version of npm available! 11.11.0 -> 11.16.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.16.0
npm notice To update run: npm install -g npm@11.16.0
npm notice

```

Figure 3. npm installation output for the API project.

Observed result: Dependencies were installed successfully. The csrf deprecation warning appears during install because the task specifically required csrf for CSRF protection; the installation completed.

```

C:\Users\Rayyan Tech\Downloads\Zainab>npm test
> cyberguard-secops-project@1.0.0 test
> npm test --prefix api

> cyberguard-secure-api@1.0.0 test
> node scripts/smoke-test.js

API not already running; starting temporary API for smoke test...
info: CyberGuard Secure API listening on port 3000 {"timestamp":"2026-06-08T23:13:48.469Z"}
Smoke test passed after temporary API start: {
  status: 'ok',
  service: 'CyberGuard Secure API',
  timestamp: '2026-06-08T23:13:48.929Z'
}
info: ::1 - - [08/Jun/2026:23:13:48 +0000] "GET /health HTTP/1.1" 200 88 "-" "node" {"timestamp":"2026-06-08T23:13:48.936Z"}

```

Figure 4. Smoke test output confirming that the API starts and returns a valid health response.

Observed result: The smoke test passed and returned status ok from the temporary API instance.

4.2 API Availability and Health Check

The root endpoint and health endpoint were opened in the browser to confirm that the API was reachable. The health route returned status ok, service name and timestamp.



```

{"status":"ok","service":"CyberGuard Secure API","message":"Use /health for health check"}

```

Figure 5. Root endpoint response from the local API.

Observed result: The API returned a valid JSON response on localhost:3000.



Figure 6. Health endpoint response from the local API.

Observed result: The /health endpoint returned status ok and confirmed that the CyberGuard Secure API was running.

4.3 API Key Authentication

The incidents endpoint was protected using API key authentication. A request without the required API key was rejected, while a request with the configured API key was accepted.

```
C:\Users\Rayyan Tech\Downloads\Zainab>curl.exe -i http://localhost:3000/incidents
HTTP/1.1 401 Unauthorized
Content-Security-Policy: default-src 'self';script-src 'self';style-src 'self' 'unsafe-inline';img-src 'self' data;;connect-src
';font-src 'self' https: data;;form-action 'self';object-src 'none';script-src-attr 'none';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: DENY
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Vary: Origin, Accept-Encoding
Access-Control-Allow-Credentials: true
RateLimit-Policy: 100;w=900
RateLimit: limit=100, remaining=93, reset=708
Content-Type: application/json; charset=utf-8
Content-Length: 34
ETag: W/"22-J4mPc5KuNFNdC0Id4KH42DAiGc"
Date: Mon, 08 Jun 2026 23:18:20 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Figure 7. Protected incidents route without API key.

Observed result: The API returned HTTP 401 Unauthorized, confirming that unauthorised access was blocked.

```
C:\Users\Rayyan Tech\Downloads\Zainab>curl.exe -i http://localhost:3000/incidents -H "x-api-key: change-this-demo-key"
HTTP/1.1 200 OK
Content-Security-Policy: default-src 'self';script-src 'self';style-src 'self' 'unsafe-inline';img-src 'self' data;;connect-src 'self';
';font-src 'self' https: data;;form-action 'self';object-src 'none';script-src-attr 'none';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: DENY
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Vary: Origin, Accept-Encoding
Access-Control-Allow-Credentials: true
RateLimit-Policy: 100;w=900
RateLimit: limit=100, remaining=92, reset=636
Content-Type: application/json; charset=utf-8
Content-Length: 16
ETag: W/"10-4h7x0SW5kDt4nX9FE7YUu0Lg/dQ"
Date: Mon, 08 Jun 2026 23:19:33 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"incidents":[]}
```

Figure 8. Protected incidents route with valid API key.

Observed result: The API returned HTTP 200 OK when the configured API key was supplied.

4.4 Rate Limiting and Failed-Login Monitoring

Repeated failed login attempts were used to test brute-force protection and alert-style monitoring. A single wrong login was rejected, repeated login attempts triggered HTTP 429, and the failed-login monitoring endpoint recorded suspicious activity.

```
C:\Users\Rayyan Tech\Downloads\Zainab>curl.exe -i -X POST http://localhost:3000/auth/login -H "Content-Type: application/json" -d '{"username":"admin","password":"wrong-password"}'
HTTP/1.1 401 Unauthorized
Content-Security-Policy: default-src 'self';script-src 'self';style-src 'self' 'unsafe-inline';img-src 'self' data;;connect-src 'self';frame-ancestors 'none';base-uri 'self';font-src 'self' https: data;;form-action 'self';object-src 'none';script-src-attr 'none';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: DENY
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Vary: Origin, Accept-Encoding
Access-Control-Allow-Credentials: true
RateLimit-Policy: 5;w=900
RateLimit: limit=5, remaining=4, reset=900
Content-Type: application/json; charset=utf-8
Content-Length: 40
ETag: W/"28-1HGtIV7not3UMayVgBivAszwTgE"
Date: Mon, 08 Jun 2026 23:20:37 GMT
Connection: keep-alive
Keep-Alive: timeout=5
{"error":"Invalid username or password"}
```

Figure 9. Wrong login attempt rejected by the authentication endpoint.

Observed result: The API returned HTTP 401 Unauthorized for an invalid username/password attempt.

```
C:\Users\Rayyan Tech\Downloads\Zainab>curl.exe -i -X POST http://localhost:3000/auth/login -H "Content-Type: application/json" -d '{"username":"admin","password":"wrong-password"}'
HTTP/1.1 429 Too Many Requests
Content-Security-Policy: default-src 'self';script-src 'self';style-src 'self' 'unsafe-inline';img-src 'self' data;;connect-src 'self';font-src 'self' https: data;;form-action 'self';object-src 'none';script-src-attr 'none';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: DENY
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Vary: Origin, Accept-Encoding
Access-Control-Allow-Credentials: true
RateLimit-Policy: 5;w=900
RateLimit: limit=5, remaining=0, reset=897
Retry-After: 897
Content-Type: application/json; charset=utf-8
Content-Length: 60
ETag: W/"3c-UAt6KPz6cMz7YPC7DdEyD1n7gr4"
Date: Mon, 08 Jun 2026 23:39:35 GMT
Connection: keep-alive
Keep-Alive: timeout=5
{"error":"Too many login attempts. Please try again later."}
```

Figure 10. Repeated failed login attempts triggering rate limiting.

Observed result: The API returned HTTP 429 Too Many Requests after repeated wrong login attempts.


```
C:\Users\Rayyan Tech\Downloads\Zainab>curl.exe -i http://localhost:3000/auth/failed-logins -H "x-api-key: change-this-demo-key"
HTTP/1.1 200 OK
Content-Security-Policy: default-src 'self';script-src 'self';style-src 'self' 'unsafe-inline';img-src 'self' data:;connect-src 'self';frame-ancestors 'none';font-src 'self' https: data:;form-action 'self';object-src 'none';script-src-attr 'none';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: DENY
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Vary: Origin, Accept-Encoding
Access-Control-Allow-Credentials: true
RateLimit-Policy: 100;w=900
RateLimit: limit=100, remaining=81, reset=480
Content-Type: application/json; charset=utf-8
Content-Length: 122
ETag: W/"7a-uboloCQ0k+aSkac7103p1SgfeII"
Date: Mon, 08 Jun 2026 23:40:24 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"suspicious_failed_logins":[{"ip":"::1","username":"admin","failed_count":10,"last_attempt":"2026-06-08T23:39:34.820Z"}]}
```

Figure 11. Failed-login monitoring output after repeated failed attempts.

Observed result: The monitoring endpoint recorded suspicious failed login activity for the admin username with failed_count shown in the response.

4.5 Security Headers, CSP and HSTS

The API response headers were checked to confirm the presence of browser security headers, including Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, X-Frame-Options and Referrer-Policy.

```
C:\Users\Rayyan Tech\Downloads\Zainab>curl.exe -I http://localhost:3000/health
HTTP/1.1 200 OK
Content-Security-Policy: default-src 'self';script-src 'self';style-src 'self' 'unsafe-inline';img-src 'self' data:;connect-src 'self';frame-ancestors 'none';base-uri 'self';font-src 'self' https: data:;form-action 'self';object-src 'none';script-src-attr 'none';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: DENY
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Vary: Origin, Accept-Encoding
Access-Control-Allow-Credentials: true
RateLimit-Policy: 100;w=900
RateLimit: limit=100, remaining=85, reset=142
Content-Type: application/json; charset=utf-8
Content-Length: 88
ETag: W/"58-CX29XTvj8CT1HF17wUHzjBoVeYU"
Date: Mon, 08 Jun 2026 23:27:46 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Figure 12. Security headers returned by the /health endpoint.

Observed result: The output shows CSP, HSTS and other hardening headers returned in the HTTP response.

5. Week 5 - Ethical Hacking and Vulnerability Fixes

Week 5 required ethical testing in a controlled environment, SQL injection protection and CSRF protection. The submitted evidence demonstrates controlled local testing of the application, rejection of invalid authentication attempts and CSRF token enforcement.

5.1 Controlled Local Security Testing

The application was tested locally using authorised HTTP requests. The tests checked unauthorised access, authentication failure, rate limiting, CSRF rejection and valid CSRF token flow. This satisfies the local test-environment aspect of the task without targeting any external system.

5.2 SQL Injection Mitigation Approach

The submitted lightweight implementation avoids raw SQL query construction in the local build.

Because the authentication and incident data handling do not rely on dynamically concatenated SQL statements, the direct SQL injection surface is avoided in this implementation.

5.3 CSRF Protection

CSRF protection was implemented and verified through token-based request testing. A request without a valid CSRF token was rejected with HTTP 403, while a request using the issued token and cookie was accepted with HTTP 200.

```
C:\Users\Rayyan Tech\Downloads\Zainab>curl.exe -c cookies.txt http://localhost:3000/csrf/token
{"csrfToken":"5oMd14Dz-fuGVb0G6tcdvEgU_8qzWM8HwhiAQ"}
```

Figure 13. CSRF token generation endpoint.

Observed result: The API returned a CSRF token and stored the related cookie for the protected request flow.

```
C:\Users\Rayyan Tech\Downloads\Zainab>curl.exe -i -X POST http://localhost:3000/csrf/protected-form -H "Content-Type: application/json" -d '{"message":"test"}'
HTTP/1.1 403 Forbidden
Content-Security-Policy: default-src 'self';script-src 'self';style-src 'self' 'unsafe-inline';img-src 'self' data;;connect-src 'self';frame-ancestors 'none';
;font-src 'self' https: data;;form-action 'self';object-src 'none';script-src-attr 'none';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: DENY
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Vary: Origin, Accept-Encoding
Access-Control-Allow-Credentials: true
RateLimit-Policy: 100;w=900
RateLimit: limit=100, remaining=89, reset=396
set-cookie: _csrf=DrktOzEtSMSfNvQ5-BBR_aZZ; Path=/
Content-Type: application/json; charset=utf-8
Content-Length: 30
ETag: W/"1e-0CKX8srdgRZaFWU+FrqO3dggrhI"
Date: Mon, 08 Jun 2026 23:23:33 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Invalid CSRF token"}
```

Figure 14. CSRF-protected endpoint without a valid token.

Observed result: The API rejected the request with HTTP 403 Forbidden and an invalid CSRF token message.

```
C:\Users\Rayyan Tech\Downloads\Zainab>curl.exe -b cookies.txt -i -X POST http://localhost:3000/csrf/protected-form -H "Content-Type: application/json" -H "x-csrf-token: nxhVj-foORaLMfapUf2LnyLBwGSG-fmx4" -d '{"message":"csrf test successful"}'
HTTP/1.1 200 OK
Content-Security-Policy: default-src 'self';script-src 'self';style-src 'self' 'unsafe-inline';img-src 'self' data;;connect-src 'self';frame-ancestors 'none';base-uri '
;font-src 'self' https: data;;form-action 'self';object-src 'none';script-src-attr 'none';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: DENY
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Vary: Origin, Accept-Encoding
Access-Control-Allow-Credentials: true
RateLimit-Policy: 100;w=900
RateLimit: limit=100, remaining=86, reset=220
Content-Type: application/json; charset=utf-8
Content-Length: 91
ETag: W/"5b-ZYuky+9uPoLoIrxSQVDHLuBZgXo"
Date: Mon, 08 Jun 2026 23:26:28 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"CSRF protected request accepted","received":{"message":"csrf test successful"}}
```

Figure 15. CSRF-protected endpoint with valid token and cookie.

Observed result: The API accepted the protected request and returned HTTP 200 OK.

6. Week 6 - Security Audit and Final Deployment Security

Week 6 required security audit reporting, deployment-readiness and final documentation. The evidence below shows the Python CLI audit component, generated report files and a readable security audit output.

6.1 Python CLI Setup and Audit Execution

The Python SecOps CLI dependencies were installed and the scanner was executed against the authorised local target. The CLI generated Markdown and JSON audit reports in the reports folder.

```
c:\Users\Rayyan Tech\Downloads\Zainab\secops_cli>py -m pip install -r requirements.txt
Collecting requests>=2.32.0 (from -r requirements.txt (line 1))
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Requirement already satisfied: rich>=13.7.0 in c:\users\rayyan tech\appdata\local\programs\python\python313\lib\site-packages (from -r requirements.txt (line 2)) (15.0.0)
Collecting python-dotenv>=1.0.1 (from -r requirements.txt (line 3))
  Downloading python-dotenv-1.2.2-py3-none-any.whl.metadata (27 kB)
Collecting charset_normalizer<4,>=2 (from requests>=2.32.0->-r requirements.txt (line 1))
  Downloading charset_normalizer-3.4.7-cp313-cp313-win_amd64.whl.metadata (41 kB)
Collecting idna<4,>=2.5 (from requests>=2.32.0->-r requirements.txt (line 1))
  Downloading idna-3.18-py3-none-any.whl.metadata (6.1 kB)
Collecting urllib3<3,>=1.26 (from requests>=2.32.0->-r requirements.txt (line 1))
  Downloading urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2023.5.7 (from requests>=2.32.0->-r requirements.txt (line 1))
  Downloading certifi-2026.5.20-py3-none-any.whl.metadata (2.5 kB)
Requirement already satisfied: markdown-it-py>=2.2.0 in c:\users\rayyan tech\appdata\local\programs\python\python313\lib\site-packages (from rich>=13.7.0->-r requirements.txt (line 2)) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in c:\users\rayyan tech\appdata\local\programs\python\python313\lib\site-packages (from rich>=13.7.0->-r requirements.txt (line 2)) (2.20.0)
Requirement already satisfied: mdurl<=0.1 in c:\users\rayyan tech\appdata\local\programs\python\python313\lib\site-packages (from markdown-it-py>=2.2.0->rich>=13.7.0->-r requirements.txt (line 2)) (0.1.2)
Downloading requests-2.34.2-py3-none-any.whl (73 kB)
Downloading charset_normalizer-3.4.7-cp313-cp313-win_amd64.whl (158 kB)
Downloading idna-3.18-py3-none-any.whl (65 kB)
Downloading urllib3-2.7.0-py3-none-any.whl (131 kB)
Downloading python-dotenv-1.2.2-py3-none-any.whl (22 kB)
Downloading certifi-2026.5.20-py3-none-any.whl (134 kB)
Installing collected packages: urllib3, python-dotenv, idna, charset_normalizer, certifi, requests
0/6 [urllib3] WARNING: The script dotenv.exe is installed in 'C:\Users\Rayyan Tech\AppData\Local\Programs\Python\Python313\Scripts' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
2/6 [idna] WARNING: The script idna.exe is installed in 'C:\Users\Rayyan Tech\AppData\Local\Programs\Python\Python313\Scripts' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
3/6 [charset_normalizer] WARNING: The script normalizer.exe is installed in 'C:\Users\Rayyan Tech\AppData\Local\Programs\Python\Python313\Scripts' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed certifi-2026.5.20 charset_normalizer-3.4.7 idna-3.18 python-dotenv-1.2.2 requests-2.34.2 urllib3-2.7.0
Activate Windows
Go to Settings to activate Windows.
```

Figure 16. Python CLI dependency installation.

Observed result: The required Python packages were installed successfully. PATH warnings are not blocking because the CLI was executed directly with py.

```
C:\Users\Rayyan Tech\Downloads\Zainab\secops_cli>py secops_cli.py --target http://localhost:3000 --out ..\reports
Auditing authorised target: http://localhost:3000/
CyberGuard Audit Findings
```

Target	Severity	Category	Finding

```
Reports saved: ..\reports\security_audit_20260609_043325.json and ..\reports\security_audit_20260609_043325.md
```

Figure 17. SecOps CLI audit execution against localhost.

Observed result: The CLI scanned the authorised localhost target and saved Markdown and JSON reports.

6.2 Generated Security Audit Reports

The generated audit files were created in the reports folder. The Markdown report was opened and reviewed as evidence of report generation.

Name	Date modified	Type	Size
completed_security_evidence_sample.md	06-Jun-26 12:17 AM	MD File	3 KB
ethical_hacking_report_template.md	06-Jun-26 12:17 AM	MD File	2 KB
final_security_audit_template.md	06-Jun-26 12:17 AM	MD File	2 KB
sample_cli_report.md	06-Jun-26 12:41 AM	MD File	1 KB
security_audit_20260609_043325.json	09-Jun-26 4:33 AM	JSON File	1 KB
security_audit_20260609_043325.md	09-Jun-26 4:33 AM	MD File	1 KB

Figure 18. Reports folder containing templates and generated audit outputs.

Observed result: The folder contains the generated security_audit Markdown and JSON files alongside project report templates.

```
security_audit_20260609_043325 - Notepad
File Edit Format View Help
# CyberGuard Security Audit Report

Generated: 2026-06-09T04:33:25

## Scan Summary

| Item | Value |
| --- | --- |
| Targets scanned | 1 |
| Findings recorded | 0 |

## Target: http://localhost:3000/

No automated findings were recorded for this target. Keep manual screenshots as final evidence.
```

Figure 19. Generated Markdown security audit report opened in Notepad.

Observed result: The report shows the scan timestamp, target scanned and findings summary.

6.3 Repository and Deployment-Readiness Artefacts

The project folder contains repository and deployment-readiness artefacts such as .github, Dockerfile, docker-compose.yml, README.md, docs and configs. These files support the required repository and deployment documentation deliverables. The final GitHub repository link should be submitted separately with the final package.

7. Requirement Completion Matrix

The following table maps the implemented and evidenced work to the internship task. The wording is intentionally limited to what is implemented or supported by the submitted project and screenshots.

Task area	Required item	Submitted implementation/evidence	Status
Week 4	Real-time monitoring / failed-login alerts	Failed-login monitoring implemented and evidenced by suspicious_failed_logins output; monitoring configuration support is included in configs.	Implemented and evidenced
Week 4	Rate limiting	express-rate-limit behaviour evidenced by HTTP 429 after repeated failed logins.	Implemented and evidenced
Week 4	CORS and API authentication	Protected endpoint rejected request without API key and accepted request with valid key.	Implemented and evidenced
Week 4	CSP and HSTS headers	HTTP header output shows Content-Security-Policy and Strict-Transport-Security.	Implemented and evidenced
Week 5	Ethical testing in test environment	Authorised localhost API testing performed using controlled HTTP requests.	Evidenced
Week 5	SQL injection prevention	Submitted local build avoids raw SQL query construction; SQL injection surface is reduced through backend design.	Addressed in implementation
Week 5	CSRF protection	CSRF token endpoint, 403 rejection and 200 valid-token acceptance are evidenced.	Implemented and evidenced
Week 6	Security audit/reporting	Python CLI generated Markdown and JSON audit reports.	Implemented and evidenced
Week 6	Dependency/repository/deployment support	README, .github, Dockerfile/docker-compose and docs/configs are included in the project structure.	Included
Week 6	Final deliverables	Final report, project ZIP, generated reports, screenshots and GitHub link are	Submission package item

		prepared/submitted separately.	
--	--	--------------------------------	--

8. Known Warnings and Clarifications

- The npm install output shows a csrf deprecation warning. This is reported transparently because the task specifically required CSRF protection using csrf middleware.
- The Python installation output shows PATH warnings for helper scripts. This did not prevent the CLI from running because it was executed directly with `py secops_cli.py`.
- The 4-5 minute video is a separate deliverable from this written report if required by the submission portal.

9. Conclusion

The CyberGuard SecOps project demonstrates a secured local API and audit-reporting workflow aligned with the Week 4 to Week 6 internship task. The evidence confirms that the API runs locally, protected endpoints enforce API key authentication, repeated failed logins trigger rate limiting, suspicious failed login activity is recorded, CSRF protection rejects invalid requests and accepts valid token-based requests, security headers are present, and the Python CLI generates audit reports. The project therefore provides a working implementation and evidence package for the core security hardening, monitoring and reporting requirements demonstrated in the supplied screenshots.

Appendix A - Key Commands Used for Evidence

Environment check:

```
node -v  
npm -v  
py --version
```

Install and test project:

```
npm install  
npm test
```

Start API:

```
npm start
```

Health check:

```
curl.exe -I http://localhost:3000/health
```

API key check:

```
curl.exe -i http://localhost:3000/incidents  
curl.exe -i http://localhost:3000/incidents -H "x-api-key: change-this-demo-key"
```

Failed-login/rate-limit test:

```
curl.exe -i -X POST http://localhost:3000/auth/login -H "Content-Type: application/json" -d  
"{\"username\":\"admin\",\"password\":\"wrong-password\"}"
```

CSRF token flow:

```
curl.exe -c cookies.txt http://localhost:3000/csrf/token  
curl.exe -b cookies.txt -i -X POST http://localhost:3000/csrf/protected-form -H "Content-Type:  
application/json" -H "x-csrf-token: TOKEN_HERE" -d "{\"message\":\"csrf test successful\"}"
```

CLI report generation:

```
cd secops_cli  
py -m pip install -r requirements.txt  
py secops_cli.py --target http://localhost:3000 --out ..\reports
```